

JOBHUNTMANAGER

README 完全解説書

採用担当者に伝わるREADMEの読み方と、
自分の言葉でプロジェクトを説明するための学習ガイド

このPDFの目的

READMEの各章について、何を伝えているか／なぜ必要か／面接でどう説明するかを、現在の実装に沿って整理します。READMEを書き換える資料ではなく、内容を理解して説明するための教材です。

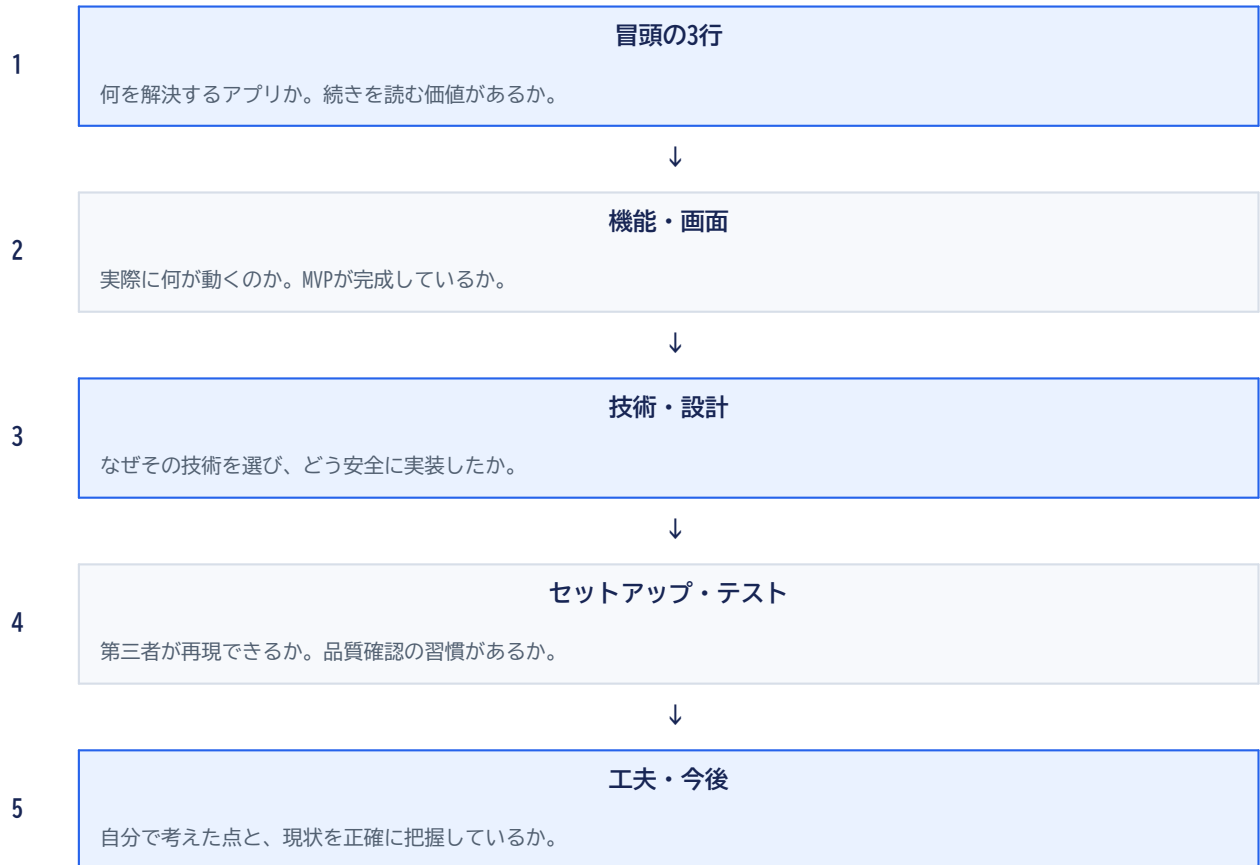
対象	内容
プロジェクト	Rails API + React TypeScript の就活管理SPA
主な画面	認証、5列カンバン、応募詳細、面接・タスク・メモ
対象README	リポジトリ直下の README.md
作成日	2026-06-23

読み終えたときのゴール：READMEを上から順に説明でき、技術選定・セキュリティ・設計上の工夫について質問されても、実装と結び付けて答えられること。

第1章 READMEは何のためにあるのか

READMEは説明書であると同時に、採用担当者が短時間でプロジェクトの価値と技術力を判断する入口です。

採用担当者が読む順番



READMEが答える4つの質問

- What: 何を作ったか
- Why: なぜ作ったか
- How: どう設計・実装したか
- Proof: 本当に動き、検証できるか

このREADMEの強み

機能一覧だけで終わらず、JWT認証、所有権、トランザクション、非同期状態管理まで説明しています。完成物と技術判断の両方が見える構成です。

面接での一言

「READMEは、最初に利用価値を伝え、その後で実装の裏付けを示す順番にしました。採用担当者が短時間でも概要を理解でき、エンジニアが読めば設計判断まで追えることを意識しています。」

第2章 冒頭3行の読み解き

READMEの冒頭は、アプリ名・分類・中心価値を最短距離で伝える場所です。

READMEに書かれている要約

原文の役割

「就職活動の応募状況を、カンバン形式で一元管理するSPA」という1文で、対象業務・主要UI・アプリ形態を同時に示しています。

```
# JOBHUNTMANAGER
```

就職活動の応募状況を、カンバン形式で一元管理するSPAです。

会社名と応募日だけで応募カードを作成でき、
選考状況・面接・タスク・メモを応募ごとに管理できます。

表現	伝わること	実装との対応
就職活動	利用場面が明確	応募・面接・タスク・メモを管理
カンバン形式	中心となる画面が分かる	5ステータスを常時表示
一元管理	情報分散を解決する	応募詳細へ関連情報を集約
SPA	画面遷移の方式が分かる	React Routerでクライアント側遷移
会社名と応募日だけ	入力負担の小ささ	簡易応募登録APIを利用

初心者ポイント

「SPA」はSingle Page Applicationの略です。最初にHTMLを読み込み、その後の画面切り替えをReactが担当します。データはRails APIからJSONで受け取ります。

説明するときの型

「誰の、どんな困りごとを、どの画面と機能で解決するアプリか」の順に話すと、技術用語だけの説明になりません。

第3章 アプリ概要と作成背景

概要は完成物を説明し、背景はなぜ作る価値があるのかを説明します。両者を分けることで話が整理されます。

アプリ概要が伝えること

- 応募単位で情報を整理する
- 5列のカンバンで進捗を把握する
- MVPでは入力項目を会社名と応募日に絞る

設計判断

データベース上はCompany、JobPosting、Applicationに分けつつ、利用者には複雑さを見せません。内部設計と入力体験を分離しています。

作成背景が伝えること

- 求人サイト・メール・カレンダー・メモへの情報分散
- 次に何をするか分からなくなる問題
- 選考の現在地が見えにくい問題

価値

単にCRUDを作ったのではなく、就職活動中の情報整理という具体的な課題から機能を逆算したことが伝わります。

課題から機能への変換

利用者の課題	採用した機能	MVPで抑えた範囲
応募状況が分散する	5列カンバン	手動並び替えは後回し
登録が面倒	会社名+応募日の簡易登録	求人詳細入力はMVP画面外
次の行動を忘れる	期限・完了状態付きタスク	横断タスク画面は今後
面接情報が散らばる	応募詳細内の面接管理	カレンダー連携は今後
企業研究を残したい	最大1万文字のメモ	タグ・添付は今後

面接での一言

「最初から多機能にせず、応募状況を迷わず把握できることと、登録の手間を減らすことをMVPの中心にしました。」

第4章 デモ・主な機能の見せ方

デモ欄は視覚的な証拠、機能欄は実装範囲の証拠です。未公開のものを「準備中」と明記する姿勢も重要です。

なぜ「準備中」と書くのか

空欄にしたり、存在しないURLを置いたりせず、現在地を正直に示すためです。未実装・未公開を実装済みのように見せないことは、ポートフォリオの信頼性につながります。

主な機能は利用者の言葉で書く

機能群	README上の説明	裏側の主な実装
認証	登録、ログイン、復元、アクセス制御	Devise、devise-jwt、AuthProvider
カンバン	5列表示、簡易登録、状態変更	Kanban API、status更新API
応募詳細	応募日編集、削除、関連情報	Applications API、ネスト表示
面接	日時・種別・状態・結果	Interviews API、型付きフォーム
タスク	期限・優先度・完了・期限超過	Tasks API、overdue表示
メモ	作成・編集・削除・文字数	Notes API、content/body変換

認証機能にセキュリティ説明を添える理由

- JWTをsessionStorageへ保存し、タブのセッション終了時に破棄しやすくする
- /auth/meで再読み込み後の認証状態を復元する
- ログイン5回/分、登録3回/時のIPレート制限で総当たり試行を抑える
- 制限超過時は429とRetry-Afterを返し、クライアントへ再試行時期を伝える

読み手が確認する点

機能名の数より、正常系だけでなく認証・失敗時・所有権まで考えているかが見られます。このREADMEはその点を具体的に書けています。

第5章 画面構成とルーティング

画面一覧は、利用者がアプリ内をどう移動するかを示す簡易サイトマップです。

画面	URL	アクセス条件	役割
ユーザー登録	/signup	未認証	アカウント作成
ログイン	/login	未認証	JWTを取得
カンバン	/kanban	認証済み	応募の一覧・追加・状態変更
応募詳細	/applications/:id	認証済み	応募と関連情報の管理

画面遷移のイメージ

1

ログイン / ユーザー登録

GuestRouteが認証済みユーザーの再訪をカンバンへ戻す



2

カンバン

ProtectedRouteが未認証アクセスをログインへ戻す



3

応募カードをクリック

React Routerで /applications/:id へ遷移



4

応募詳細

応募日、面接、タスク、メモを同じ文脈で管理

なぜ求人画面をMVP
の導線から外したの
か

利用者が会社・求人・応募を順番に作る操作は負担が大きいためです。APIは残しながら、MVP画面ではカンバンからの簡易登録を中心にしました。

設計書との関係

READMEは全体像、docs/spec.mdはMVP仕様、docs/api_design.mdは通信契約、docs/database_design.mdは永続化設計を担当します。詳しさの階層を分けています。

第6章 使用技術を説明できるようになる

技術一覧では名前を並べるだけでなく、このアプリのどの責務を担当しているかを理解することが重要です。

Backend

技術	このアプリでの役割
Rails API	JSON APIと業務ルール
PostgreSQL	関連データと制約の保存
Devise	ユーザー認証の土台
devise-jwt	JWT発行・失効
Rack::Attack	認証試行の制限
rack-cors	SPAオリジンの許可

Frontend

技術	このアプリでの役割
React	状態に応じたUI構築
TypeScript	APIデータの型安全性
Vite	開発・ビルド環境
Tailwind CSS	一貫した見た目
Axios	API通信と共通処理
React Router	SPA内の画面遷移

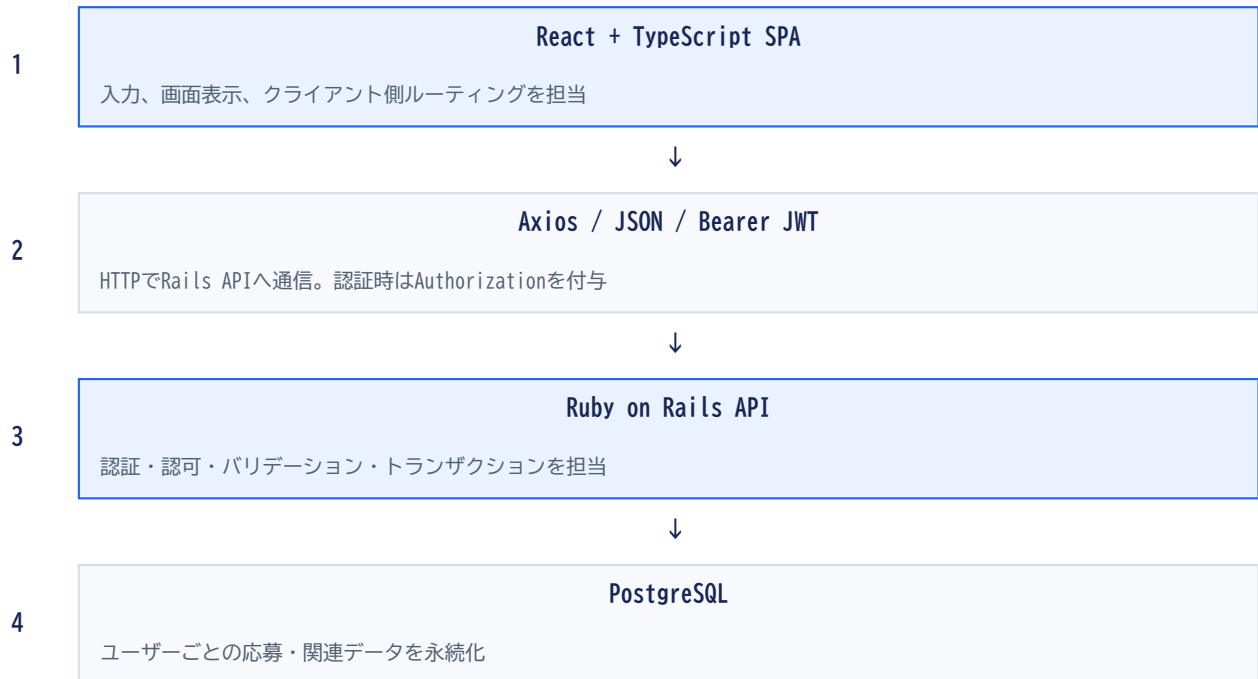
技術選定を説明する公式

課題 → 選択 → 効果 例：「APIレスポンスのフィールド違いを開発時に検出したかったためTypeScriptを採用し、ApplicationやTaskなどの型を機能単位で定義しました。」

質問	短い回答例
なぜRails API？	認証・関連・検証をRailsへ集約し、ReactとはJSONで分離するためです。
なぜReact？	カンバンやフォームの状態変化を、部品単位で構築しやすいためです。
なぜAxios？	JWT付与と401処理をinterceptorへ集約しやすいためです。
なぜReduxなし？	共有するグローバル状態が主に認証だけで、Contextで十分だからです。

第7章 システム構成とデータの流れ

READMEの構成図は、ブラウザ・API・DBの責務が分離されていることを一目で示します。



簡易応募登録のデータフロー

1. React: 会社名と応募日を入力
2. POST /api/v1/kanban/applications
3. Rails: `current_user` を認証
4. `transaction` 内で `Company` を再利用/作成
5. `JobPosting` と `Application` を作成
6. 作成したKanbanカードをJSONで返す
7. React: 応募済み列へ即時追加

なぜ処理をRails側へ寄せるのか

フロントから3つのAPIを順番に呼ぶと、途中失敗で`Company`だけ残る可能性があります。Railsのトランザクションなら一連の作成を成功または全取消のどちらかにできます。

セキュリティの中心

APIはリクエストの`user_id`を信用しません。認証済みの`current_user`を起点に検索・作成するため、他ユーザーのIDを推測しても操作できない構成です。

第8章 DB設計概要と所有権

ER図はテーブル一覧ではなく、データ同士の責任関係を示す図です。

関連の中心

親	子	意味
User	Company / JobPosting / Application	利用者が直接所有するデータ
Company	JobPosting	企業に求人が属する
JobPosting	Application	求人に対する応募
Application	Interview / Task / Note	応募の文脈に詳細情報を集約

所有権の判定

```
companies / job_postings / applications
└─ user_id を持つ

interviews / tasks / notes
└─ user_id を持たない
   application → user の関連をたどって所有者を判定
```

利点

子テーブルへuser_idを重複保存しないため、application.user_idとnote.user_idが食い違う不整合を防げます。

実装上の注意

Interview.find(params[:id])ではなく、current_userのapplicationsに関連する範囲から検索する必要があります。

削除方針

- Application削除時はInterview・Task・Noteを連鎖削除する
- Company・JobPostingは子データがある場合に削除を拒否する
- 404を返すことで、他ユーザーのデータが存在するかどうかも明かさない

面接での一言

「直接所有するテーブルだけuser_idを持たせ、面接・タスク・メモはApplication経由で認可しています。所有権情報の重複を避け、検索時もcurrent_userのスコープを使っています。」

第9章 API概要の読み方

API表は、ReactとRailsの間で交わす通信契約を一覧化したものです。

HTTP	意味	このアプリの例
GET	取得	カンバン、応募詳細、関連一覧
POST	新規作成	登録、ログイン、簡易応募、面接等
PATCH	一部更新	応募日、ステータス、タスク等
DELETE	削除	ログアウト、応募、面接、タスク、メモ

代表的な通信

```
POST /api/v1/kanban/applications
Authorization: Bearer
Content-Type: application/json

{
  "application": {
    "company_name": "株式会社サンプル",
    "applied_on": "2026-06-23"
  }
}
```

成功レスポンス

```
{
  "data": {
    "id": 1,
    "company_name": "株式会社サンプル",
    "status": "applied"
  }
}
```

422エラー

```
{
  "error": {
    "code": "validation_error",
    "message": "入力内容を確認してください",
    "details": { ... }
  }
}
```

通常更新と状態更新 を分ける理由

応募日の編集はPATCH /applications/:id、カンバンの状態変更はPATCH /applications/:id/statusへ分けています。許可するStrong Parametersを狭くし、意図しない項目変更を防ぎます。

第10章 セットアップ手順の意味

セットアップ手順は、第三者がREADMEだけを見て動作確認できるかを左右します。

全体の順番



なぜ必要バージョンを書くのか

- Ruby 3.3.10 : GemやRailsの動作条件を揃える
- Node.js ^20.19.0 または >=22.12.0 : Viteの実行条件を満たす
- PostgreSQL : 事前起動し、DB_USERNAMEにDB作成権限を持たせる

```
git clone https://github.com/konohq/JOBHUNTMANAGER.git
cd JOBHUNTMANAGER

cd backend
Copy-Item .env.example .env
bundle install
bundle exec rails db:prepare
bundle exec rails server
```

db:prepareとは

データベースがなければ作成し、schemaやmigrationを使って利用可能な状態へ整えるRailsコマンドです。初回セットアップで手順を簡潔にできます。

よくあるつまづき

RailsとViteは別ポートで起動します。Railsはlocalhost:3000、Reactはlocalhost:5173です。両方を別ターミナルで起動します。

第11章 環境変数と秘密情報

環境変数の章は、動作設定だけでなく『何をGitへ置いてはいけないか』を明確にするセキュリティ説明です。

変数	秘密か	理由
DB_USERNAME	環境による	ユーザー名自体より権限管理が重要
DB_PASSWORD	秘密	DBへ接続できる認証情報
DEVISE_JWT_SECRET_KEY	秘密	JWT署名を偽造されないために必要
FRONTEND_ORIGIN	通常は公開可	CORSで許可するWebサイトURL
VITE_API_BASE_URL	公開情報	ビルド後のJavaScriptから見えるAPI URL

最重要

VITE_で始まる値は秘密にできません。ViteがフロントエンドのJavaScriptへ埋め込むため、利用者のブラウザから確認できます。API URLだけを置き、パスワード・JWT秘密鍵・APIキーは置きません。

.env と .env.example の違い

.env

実際のローカル値を持つ。Git管理対象外。DBパスワードやJWT秘密鍵を含められる。

.env.example

必要な変数名とサンプル値だけを示す。Git管理対象。秘密情報は書かない。

```
# backend/.env
DB_PASSWORD=<実際のパスワード>
DEVISE_JWT_SECRET_KEY=<十分に長いランダム値>

# frontend/.env
VITE_API_BASE_URL=http://localhost:3000
```

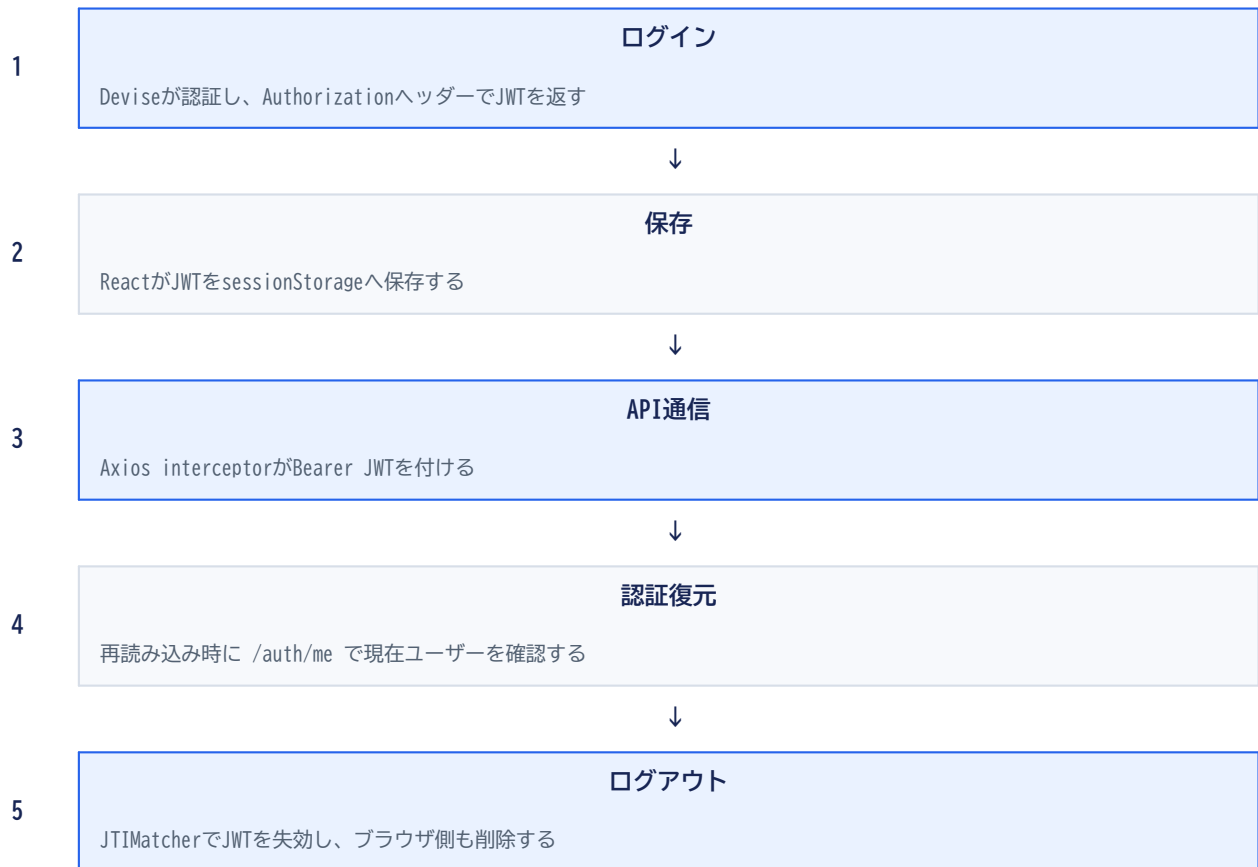
READMEの役割

値そのものではなく、変数名・用途・安全な設定方法を示します。JWT秘密鍵の生成コマンドまで書くことで、弱い固定文字列を使う事故を防ぎます。

第12章 認証・レート制限・エラー

セキュリティ機能は『導入した』だけでなく、どの攻撃や失敗をどう扱うかを説明できることが大切です。

JWT認証の流れ



レート制限

対象	上限	超過時
ログイン	同一IPで1分5回	429 + Retry-After
ユーザー登録	同一IPで1時間3回	429 + Retry-After

status	意味	フロントの扱い
401	認証できない	認証状態を解除しログインへ
404	対象が見つからない/所有外	詳細を隠してエラー表示
422	入力値が不正	該当入力欄へ詳細を表示
429	試行回数超過	時間をおいて再試行
500	サーバー内部エラー	JWTを安易に消さず再試行可能にする

sessionStorageの注意

JavaScriptから読めるためXSS対策が必要です。Reactの通常の文字列描画を使い、危険なHTML挿入を避け、依存関係を更新し、秘密情報をフロントへ置かないことが前提です。

第13章 テスト・静的解析の読み方

READMEに検証コマンドを書くことで、『どの観点で品質を確認しているか』を第三者が再現できます。

コマンド	確認すること
<code>bundle exec rails test</code>	モデル・API・認証・所有権などの動作
<code>bundle exec rubocop</code>	Rubyコードのスタイルと問題パターン
<code>bundle exec brakeman --no-pager</code>	Rails特有の脆弱性候補
<code>bundle exec rails zeitwerk:check</code>	ファイル名と定数の読み込み整合性
<code>npm run test</code>	API関数やフロントの振る舞い
<code>npm run build</code>	TypeScriptを含む本番ビルド
<code>npm run lint</code>	React・TypeScriptの静的解析
<code>git diff --check</code>	余分な空白や改行上の差分問題

検査は役割が違う

テスト

期待した入力に対して期待した出力になるかを確認します。所有外データの404や、重複応募の422も対象です。

静的解析

実行せずコードを調べ、書き方・危険な構造・読み込みミスを検出します。

READMEで断言しすぎない

GitHub Actionsのworkflowが自動実行される配置でなければ、『CI稼働中』とは書きません。実際に確認できるローカルコマンドを掲載する方が正確です。

面接での回答例

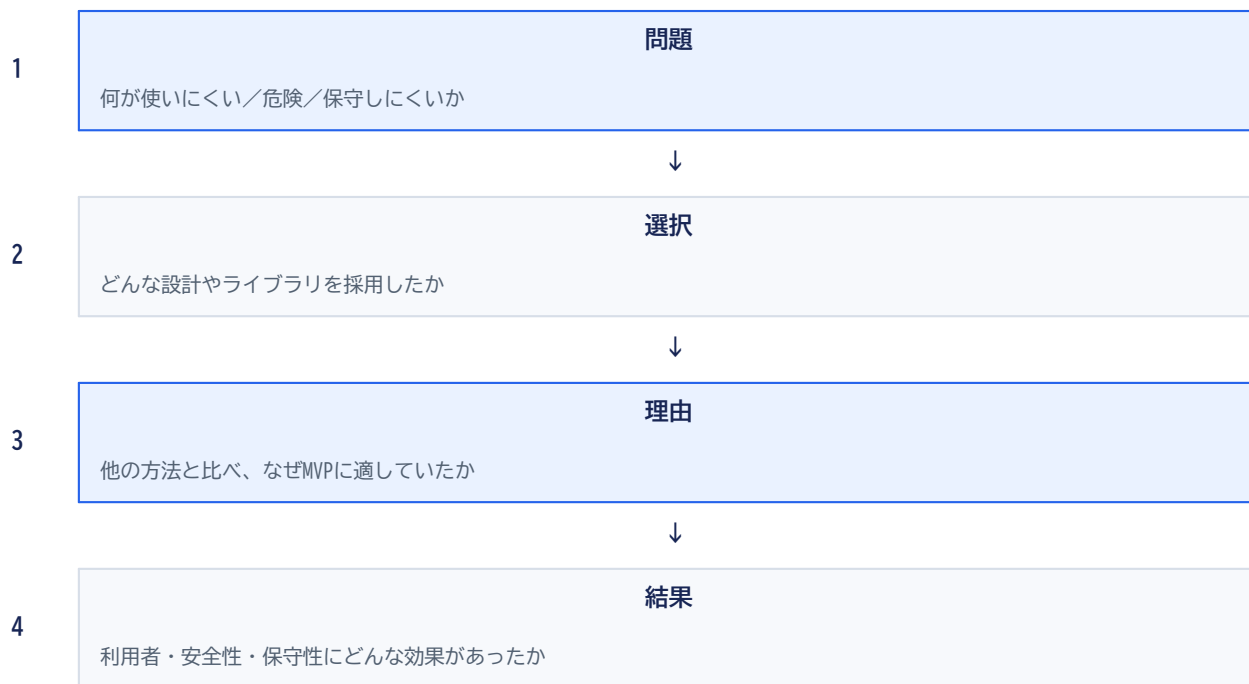
「APIテストで認証・所有権・バリデーションを確認し、RuboCopとESLintでコード品質、BrakemanでRailsの脆弱性候補、buildとZeitwerkで本番ビルド・定数読み込みを確認しています。」

第14章 工夫した点は技術判断の証拠

この章は、実装した機能より一段深く『どんな問題に気づき、どう設計したか』を示します。

工夫	問題	解決
簡易応募登録	3種類の入力面倒	会社名と応募日だけを受け、Railsで一括作成
トランザクション	途中失敗で中途半端に残る	全成功または全ロールバック
所有権スコープ	ID推測による他人の操作	current_user起点で検索
JWT復元	再読み込みで状態が消える	/auth/meでユーザーを復元
5列固定API	空列のUI処理が複雑	全statusを空配列込みで返す
Setで通信管理	並行操作でloadingが競合	操作中IDを複数保持
API/UI分離	画面が通信詳細で肥大化	features単位にAPIと型を分離

工夫を話す4ステップ



特に説明価値が高い点

簡易応募登録は、UX・DB正規化・トランザクション・重複制御を1つの題材で説明できます。このプロジェクトの代表的な設計判断です。

第15章 今後の追加予定と面接用まとめ

今後の機能は未完成の告白ではなく、MVPの境界を理解し、次の優先順位を考えている証拠です。

今後の機能を分類する

- 発見性: 検索、絞り込み、ページネーション
- 行動管理: 横断タスク、カレンダー、通知
- 情報拡張: タグ、添付、CSV
- 操作性: D&D;、手動並び替え
- アカウント: パスワード再設定、退会
- 公開: レスポンシブ改善、本番デプロイ

公開時の表現

未実装機能は必ず「今後の追加予定」に置きます。主な機能欄へ混ぜないことで、現在動く範囲が明確になります。

優先順位の考え方

利用頻度、利用者への効果、実装コスト、セキュリティリスクの4点で判断すると説明しやすくなります。

1分で説明する例

「JOBHUNTMANAGERは、就職活動の応募状況を5列のカンバンで一元管理するSPAです。応募登録の手間を減らすため、会社名と応募日だけでカードを作成できます。Rails APIとReactを分離し、JWT認証、current_userを起点にした所有権制御、トランザクションによる安全な一括作成を実装しました。応募詳細では面接・タスク・メモを管理できます。MVPでは状況把握と入力のしやすさを優先し、検索やドラッグ&ドロップは今後の機能に分けています。」

よく聞かれる質問

質問	回答の軸
一番工夫した点は？	簡易応募登録とトランザクション
セキュリティは？	JWT、所有権、Strong Parameters、Rate Limit
MVPで削ったものは？	求人画面中心の導線、D&D;、検索、横断画面
次に改善するなら？	デプロイ、画面証拠、アクセシビリティ、E2E

最終チェック

READMEの各節について「何を書いたか」だけでなく「なぜ必要か」「実装のどこに対応するか」を説明できれば、ポートフォリオを自分の言葉で語る状態です。

— End of Guide —